

A Project on Parallelising a Jacobi-2D Algorithm Using OpenMP DOALL Parallelism and Comparing the Execution Time with that of Sequential Execution

Team Name : Silicon Strikers

Aastik Bansal 23114002

Suprajeet Suman 23114097

Ridham Dave 23114087

Utkarsh Raj 23114102

Harshit Kumar Meena 23114037

Table of Contents :

1. Introduction
 - Jacobi-2D Stencil Algorithm
2. Algorithm Details
 - Problem Setup
 - Jacobi Stencil Update Rule
 - Iteration Steps
 - Convergence Criteria
 - Complexity Analysis
 - Pseudocode and Explanation of `collapse(2)`
3. Comparing Sequential and Parallel Implementations
 - 3.1 Sequential Implementation
 - 3.2 Parallel Implementation using OpenMP
 - 3.3 Validation of Parallelization
 - 3.4 Code Implementation for Time Comparisons
4. Performance Analysis
 - 4.1 Execution Time
 - 4.2 Experimental Setup
 - 4.3 Results and Graphs
 - 4.4 Analysis of Performance Gains
 - 4.5 Scheduling Results (Static, Guided, Dynamic)
 - 4.6 Comparison: Column-wise vs Row-wise Iteration
 - 4.7 Tiling for Improved Cache Utilization
 - 4.8 Error Matrix: Limitations in Parallelizing Iterations
5. References

Introduction :

Jacobi-2D Stencil Algorithm:

Purpose:

The Jacobi-2D Stencil method is an iterative algorithm used for solving Laplace's equations. The term stencil refers to a fixed pattern of points (neighboring cells) used to update each grid point. It is widely used in scientific computing for simulating processes like heat diffusion or fluid flow.

Algorithm Overview:

1. Problem Setup:

- Consider a 2D grid u of size $N \times N$ representing the solution domain.
- Given an initial guess for the solution $u^{(0)}$.
- Define boundary conditions and the convergence criterion ϵ .

2. Jacobi Stencil Update Rule:

- For each interior grid point (i,j) , the new value $u_{\text{new}}[i][j]$ is computed as the average of its neighboring points:

$$u_{\text{new}}[i][j] = \frac{1}{5}(u[i-1][j] + u[i+1][j] + u[i][j-1] + u[i][j+1] + u[i][j])$$

- Here, each point is updated based on its four immediate neighbors (up, down, left, right).

3. Iteration steps:

- **Step 1:** Initialize $u^{(0)}$ with the initial guess (e.g., zeros or boundary values).
- **Step 2:** For each iteration k :
 - For each interior grid point (i,j) :

$$u_{\text{new}}[i][j] = \frac{1}{5}(u[i-1][j] + u[i+1][j] + u[i][j-1] + u[i][j+1] + u[i][j])$$

- Update $u[i][j]$ with $u_{\text{new}}[i][j]$.
- **Step 3:** If the error is less than ϵ , the solution has converged; otherwise, repeat Step 2

4. Convergence:

The Jacobi method converges if the error decreases below the defined tolerance level. The convergence

is guaranteed for diagonally dominant systems or when applied to certain types of partial differential equations like the Laplace equation.

Pseudocode:

```
Jacobi2DStencil(u, maxIterations, tolerance):
```

```
    N = size(u)
```

```
    u_new = copy(u)
```

```

for k = 1 to maxIterations:
    error = 0
    parallel for collapse(2)
    for i = 1 to N-1:
        for j = 1 to N-1:
            u_new[i][j] = 0.2 * (u[i-1][j] + u[i+1][j] + u[i][j-1] + u[i][j+1] + u[i][j])

            # Calculate the maximum error
            error = max(error, abs(u_new[i][j] - u[i][j]))

        # Update the solution grid
        u = copy(u_new)

    # Check for convergence
    if error < tolerance:
        break

return u

```

Explanation of collapse(2)

1. Collapse Clause: The `collapse(2)` clause combines the two nested loops over `i` and `j` into a single loop, which is then parallelized. This results in better workload distribution across threads, especially on larger grids.
2. DOALL Efficiency: By collapsing, we can achieve finer-grained parallelism, which can lead to improved utilization of the hardware resources and better performance in terms of execution time.

Applications:

- Commonly used in solving heat distribution problems and fluid dynamics.
- It is a fundamental method in finite difference schemes for numerical solutions of partial differential equations.

Complexity:

- **Time Complexity:** $O(N^2 \times K)$, Where **N** is the Grid size and **K** is the number of iterations
- **Space Complexity:** $O(N^2)$ for the grid.

Comparing the time of sequential and parallel processing of Jacobi 2D algorithm:

1. Introduction

It is particularly effective for large-scale problems where direct methods are computationally expensive. In this project, we implemented a 2D Jacobi stencil algorithm. The

focus of this project is to compare the performance of sequential and parallel implementations of the Jacobi method using OpenMP.

2. Objective

The objective of this project is to:

- Implement the Jacobi 2D stencil algorithm in both sequential and parallel modes using OpenMP.
- Measure and compare the performance of both implementations.
- Analyze the speedup achieved by the parallel implementation.

3. Methodology

3.1 Sequential Implementation

In the sequential version:

- Two matrices **A** and **B** of size $N \times N$ are dynamically allocated.
- The matrix **A** is initialized with boundary values 140 and 10 and other values 30.
- For each iteration, a nested loop updates the values of matrix **B** using the Jacobi formula based on values from matrix **A**.
- After processing each element, matrices **A** and **B** are swapped.
- The time taken for this computation is recorded using `omp_get_wtime()`.
- The file that have the source Code for Sequential Execution is [sequential.cpp](#)

3.2 Parallel Implementation using OpenMP

In the parallel version:

- OpenMP is used to parallelize the nested loops that compute the Jacobi updates.
- `#pragma omp parallel for collapse(2)` is used to allow parallel processing of both row and column indices simultaneously.
- A barrier (`#pragma omp barrier`) ensures synchronization between threads before matrices **A** and **B** are swapped.
- The time taken (`t2`) for the parallel execution is recorded for comparison.
- The file that have the source Code for Parallel Execution is [jacobi_parallel.cpp](#)

3.3 Validation :

This validates the correctness of parallelization in the Jacobi iterative method by comparing the results of the parallel and sequential versions. The matrix is initialized with boundary values, and two computation methods are executed:

1. Parallel Computation:

The Jacobi method is executed with OpenMP parallelization, updating the matrix based on neighboring values from the matrix.

To improve performance further, we can apply the `collapse(2)` clause in the DOALL parallelization, which allows both loops over **i** and **j** to be collapsed into a single parallel loop. This change distributes the workload more evenly across threads, enhancing load balancing and reducing synchronization overhead.

After each iteration, the maximum difference between consecutive iterations is checked to ensure convergence.

2. Sequential Computation:

The Jacobi method is then executed sequentially without any parallel optimization to calculate the matrix values again.

3. Validation Check:

The matrices resulting from parallel and sequential computations are compared element-wise.

The difference between corresponding elements is printed to verify that the results are nearly identical, confirming the correctness of the parallel implementation.

This comparison ensures that the parallel optimization does not alter the final results, validating the reliability of the applied parallelism.

File name : [matrix.cpp](#)

Code and Output of validation:

```
chmod +x bash4.sh
./bash4.sh
```

```
suppa@HP-Victus-SS:~/Documents/COA project/conda$ chmod +x bash4.sh
suppa@HP-Victus-SS:~/Documents/COA project/conda$ ./bash4.sh
Before any operations :
10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000
10.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 140.000000
10.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 140.000000
10.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 140.000000
10.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 140.000000
10.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 140.000000
10.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 140.000000
10.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 140.000000
10.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 140.000000
10.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 140.000000
10.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000
matrix with parallelism
10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000
10.000000 13.504944 17.017049 20.630839 24.615657 29.557614 36.718946 49.105543 74.940471 140.000000
10.000000 17.017049 23.959340 30.926937 38.315458 46.937159 58.248966 74.789738 100.670701 140.000000
10.000000 20.630839 30.926937 40.870327 50.859687 61.704245 74.618331 91.184464 112.979588 140.000000
10.000000 24.615657 38.315458 50.859687 62.653320 74.506453 87.427719 102.418561 120.099572 140.000000
10.000000 29.557614 46.937159 61.704245 74.506453 86.359586 98.272277 111.040262 125.041529 140.000000
10.000000 36.718946 58.248966 74.618331 87.427719 98.272277 108.366336 118.506494 129.067696 140.000000
10.000000 49.105543 74.789738 91.184464 102.418561 111.040262 118.506494 125.620137 132.759195 140.000000
10.000000 74.940471 100.670701 112.979588 120.099572 125.041529 129.067696 132.759195 136.375998 140.000000
10.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000
matrix without parallelism
10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000
10.000000 13.504944 17.017049 20.630839 24.615657 29.557614 36.718946 49.105543 74.940471 140.000000
10.000000 17.017049 23.959340 30.926937 38.315458 46.937159 58.248966 74.789738 100.670701 140.000000
10.000000 20.630839 30.926937 40.870327 50.859687 61.704245 74.618331 91.184464 112.979588 140.000000
10.000000 24.615657 38.315458 50.859687 62.653320 74.506453 87.427719 102.418561 120.099572 140.000000
10.000000 29.557614 46.937159 61.704245 74.506453 86.359586 98.272277 111.040262 125.041529 140.000000
10.000000 36.718946 58.248966 74.618331 87.427719 98.272277 108.366336 118.506494 129.067696 140.000000
10.000000 49.105543 74.789738 91.184464 102.418561 111.040262 118.506494 125.620137 132.759195 140.000000
10.000000 74.940471 100.670701 112.979588 120.099572 125.041529 129.067696 132.759195 136.375998 140.000000
10.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000
Serial - Parallel :
0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
suppa@HP-Victus-SS:~/Documents/COA project/conda$
```

3.4 Code Implementation for time comparisons:

The two files `jacobi_parallel.cpp` and `sequential.cpp` can be run simultaneously using

bash.sh

chmod +x bash.sh

./bash.sh

```
suppa@HP-Victus-SS:~/Documents/COA project/coa$ ./bash.sh
For 2 threads
Running Jacobi 2D for N=10
Time in parallel running of this code is 0.000284
Running Jacobi 2D for N=100
Time in parallel running of this code is 0.005473
Running Jacobi 2D for N=10000
Time in parallel running of this code is 29.765233

For 4 threads
Running Jacobi 2D for N=10
Time in parallel running of this code is 0.000363
Running Jacobi 2D for N=100
Time in parallel running of this code is 0.003278
Running Jacobi 2D for N=10000
Time in parallel running of this code is 17.629962

For 8 threads
Running Jacobi 2D for N=10
Time in parallel running of this code is 0.008583
Running Jacobi 2D for N=100
Time in parallel running of this code is 0.018115
Running Jacobi 2D for N=10000
Time in parallel running of this code is 14.739962

For 12 threads
Running Jacobi 2D for N=10
Time in parallel running of this code is 0.000495
Running Jacobi 2D for N=100
Time in parallel running of this code is 0.001731
Running Jacobi 2D for N=10000
Time in parallel running of this code is 14.438564

For 24 threads
Running Jacobi 2D for N=10
Time in parallel running of this code is 0.004307
Running Jacobi 2D for N=100
Time in parallel running of this code is 0.005755
Running Jacobi 2D for N=10000
Time in parallel running of this code is 13.020620

Sequential
Running Jacobi 2D for N=10
Time in sequential running of this code is 0.000036
Running Jacobi 2D for N=100
Time in sequential running of this code is 0.005342
Running Jacobi 2D for N=10000
Time in sequential running of this code is 46.765555
suppa@HP-Victus-SS:~/Documents/COA project/coa$
```


4. Performance Analysis

4.1 Execution Time

We recorded the execution times for both sequential (t_1) and parallel (t_2) implementations. The performance improvement was calculated as follows:

Speedup = t_1/t_2

4.2 Experimental Setup

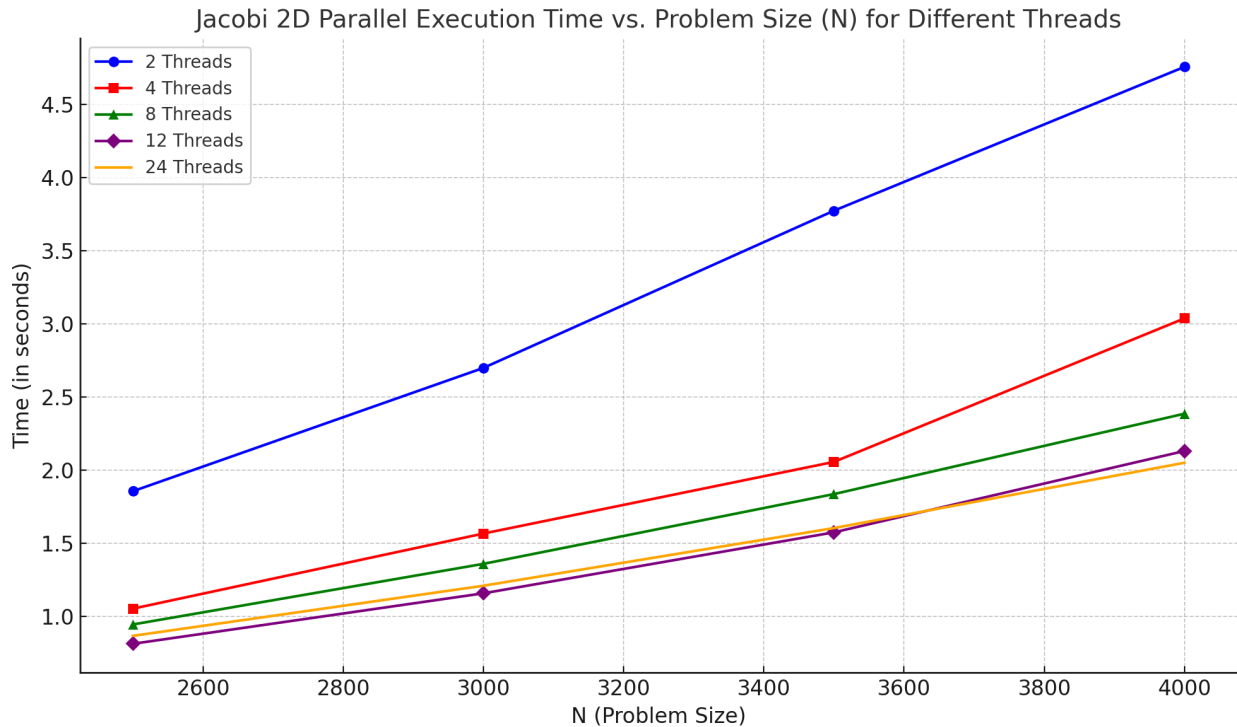
- System Configuration: 12 cores, L1 cache: 704 KB, L2 cache: 7.0 MB, L3 cache: 12.0 MB.
- Compiler: g++ with OpenMP support (`-fopenmp` flag)
- Matrix Sizes Tested: $N = 10, 100, 10000$
- Number of Iterations: 100 (defined by `MAX_ITERATIONS`)

```
suppa@HP-Victus-SS:~$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 12
On-line CPU(s) list:    0-11
Vendor ID:              GenuineIntel
Model name:             12th Gen Intel(R) Core(TM) i5-12450H
CPU family:             6
Model:                  154
Thread(s) per core:     2
Core(s) per socket:     8
Socket(s):              1
Stepping:               3
CPU max MHz:            4400.0000
CPU min MHz:            400.0000
BogoMIPS:               4992.00
```

```
Caches (sum of all):
  L1d:          320 KiB (8 instances)
  L1i:          384 KiB (8 instances)
  L2:           7 MiB (5 instances)
  L3:          12 MiB (1 instance)
```

4.3 Results:

Matrix Size (N x N)	Sequential Time (s)	Parallel Time (2 Threads) (s)	Speedup (2 Threads)	Parallel Time (4 Threads) (s)	Speedup (4 Threads)	Parallel Time (8 Threads) (s)	Speedup (8 Threads)	Parallel Time (12 Threads) (s)	Speedup (12 Threads)	Parallel Time (24 Threads) (s)	Speedup (24 Threads)
10 x 10	0.000036	0.000284	0.1268	0.000363	0.0992	0.000583	0.0617	0.000495	0.0727	0.000437	0.0823
100 x 100	0.005342	0.005473	0.976	0.003278	1.630	0.001815	2.942	0.001731	3.086	0.005755	0.928
10,000 x 10,000	46.765555	29.765233	1.570	17.629962	2.653	14.739962	3.173	14.438564	3.239	13.020620	3.592



Graphs:

- We have Plotted a 3D graph for the data that we Gathered using a Python code and the Source code is in the file [graph.py](#).
- The python file can be executed by running the command following in the Terminal
`python3 -u "graph.py"`

4.4 Analysis

- Performance Gains: The parallel implementation showed significant speedup for larger matrix sizes. The speedup increases with the size of the matrix because the overhead of thread creation and management becomes less significant relative to the computational load.
- Scalability: The use of `collapse(2)` in OpenMP effectively distributes the computational load across available threads, enhancing scalability on multi-core systems.
- Load Balancing: The Jacobi 2D stencil computation is inherently parallelizable as each grid point update is independent of others, resulting in efficient load balancing.

4.5 Results After Scheduling

- Static Scheduling: Static Scheduling is a way to distribute loop iterations among threads in a fixed pattern, where each thread is assigned a predetermined, contiguous chunk of iterations. This assignment is decided before the loop begins, based on the total number of iterations and threads.

The file that uses static Scheduling is [schedule_static.cpp](#).

- Guided Scheduling: This scheduling policy is similar to a dynamic schedule, except that the chunk size changes as the program runs.

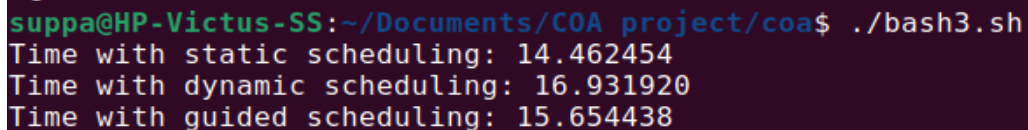
The file that uses guided scheduling is [schedule_guided.cpp](#).

- Dynamic Scheduling: Dynamic scheduling distributes loop iterations among threads at runtime, assigning chunks of iterations to threads as they become available

The file that uses dynamic scheduling is [schedule_dynamic.cpp](#).

All of the 3 files are ran simultaneously using a bash script which is in the file [bash3.sh](#). The results are as follows

```
chmod +x bash3.sh
./bash3.sh
```

A terminal window with a dark purple background. The prompt is 'suppa@HP-Victus-SS:~/Documents/COA project/coa\$'. The command './bash3.sh' has been executed, resulting in three lines of output: 'Time with static scheduling: 14.462454', 'Time with dynamic scheduling: 16.931920', and 'Time with guided scheduling: 15.654438'.

```
suppa@HP-Victus-SS:~/Documents/COA project/coa$ ./bash3.sh
Time with static scheduling: 14.462454
Time with dynamic scheduling: 16.931920
Time with guided scheduling: 15.654438
```

4.6 Results After Iterating the array column wise instead of row-wise:

- When we iterate the array column wise, The number of Cache Miss increases as this approach does not exploit the spatial locality. So the Time compared to that of row wise iteration increases.
- The file that has the source Code for Execution using column wise iteration is [memory_miss.cpp](#).
- The file that has the source Code for Execution using row wise iteration is [memory_hit.cpp](#).
- Both files were ran using a bash script that is in the file [bash2.sh](#). The result are :

```
chmod +x bash2.sh
./bash2.sh
```

```
suppa@HP-Victus-SS:~/Documents/COA project/coa$ ./bash2.sh
Exploiting spatial locality of cache
Parallel Time while fetching matrix row wise: 6.693421 seconds
Parallel Time while fetching matrix column wise: 15.232523 seconds
suppa@HP-Victus-SS:~/Documents/COA project/coa$
```

4.7 Results after using tiling to implement parallelism:

In our parallel Jacobi solver, we implemented **tiling** to improve cache utilization by processing smaller sub-blocks of the matrix. This technique aims to reduce cache misses and enhance data locality. However, our experiments showed that tiling did **not** significantly improve performance.

Possible reasons include:

1. **Overhead of Tiling:** The additional loop structure and tile management introduced extra overhead, which could outweigh the cache benefit.
2. **Efficient Cache Usage Without Tiling:** The cache may have already been effectively utilized in the non-tiled version, reducing the potential gains from tiling.
3. **Thread Synchronization Costs:** Increased synchronization between threads processing different tiles may have introduced contention, affecting performance.

Overall, the lack of improvement suggests that tiling's benefits were negated by its overhead in this specific implementation.

File name : [tiling.cpp](#)

```
chmod +x bash5.sh
./bash5.sh
```

```
suppa@HP-Victus-SS:~/Documents/COA project/coa$ ./bash5.sh
Time for Jacobi without tiling: 12.471517 seconds
Time for Jacobi with tiling: 16.263566 seconds
Performance improvement with tiling: -30.41%
suppa@HP-Victus-SS:~/Documents/COA project/coa$
```

4.8 The Error Matrix:

- When we tried to Parallelise the number of iterations along with the matrix calculations, We were getting wrong output as the array was updated simultaneously by different iterations which led to overwriting errors.
- This implies that we cannot optimize this algorithm any further using any more parallelisation.
- The file that has the error matrix is [matrix_error.cpp](#).

- The Error Matrix is:

```
g++ -fopenmp matrix_error.cpp -o matrix_error
./matrix_error 10
```

```
suppa@HP-Victus-SS:~/Documents/COA project/coa$ g++ -fopenmp matrix_error.cpp -o matrix_error
suppa@HP-Victus-SS:~/Documents/COA project/coa$ ./matrix_error 10
Before any operations :
10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000
10.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 140.000000
10.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 140.000000
10.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 140.000000
10.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 140.000000
10.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 140.000000
10.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 140.000000
10.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 140.000000
10.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 140.000000
10.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 30.000000 140.000000
10.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000
matrix with parallelism
10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000
10.000000 13.550877 17.104438 20.749955 24.752617 29.696021 36.841877 49.197649 74.989918 140.000000
10.000000 17.104607 24.125779 31.153620 38.575905 47.200174 58.482420 74.964550 100.764496 140.000000
10.000000 20.750447 31.154107 41.179483 51.214633 62.062448 74.936077 91.422256 113.107107 140.000000
10.000000 24.753450 38.576965 51.215319 63.061355 74.919797 87.792557 102.691450 120.245839 140.000000
10.000000 29.697066 47.201635 62.063689 74.918621 86.775054 98.640423 111.315495 125.188987 140.000000
10.000000 36.842916 58.483943 74.937511 87.793533 98.640844 108.692777 118.750444 129.198342 140.000000
10.000000 49.198458 74.965766 91.423460 102.692355 111.316005 118.750625 125.802510 132.856830 140.000000
10.000000 74.990342 100.765142 113.107762 120.246353 125.189303 129.198486 132.856868 136.428272 140.000000
10.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000
matrix without parallelism
10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000 10.000000
10.000000 22.000000 24.400000 24.880000 24.976000 24.995200 24.999040 24.999808 46.999962 140.000000
10.000000 24.400000 27.760000 28.528000 28.700800 28.739200 28.747648 28.749491 55.149891 140.000000
10.000000 24.880000 28.528000 29.411200 29.622400 29.672320 29.683994 29.686697 56.967318 140.000000
10.000000 24.976000 28.700800 29.622400 29.848960 29.904256 29.917650 29.920869 57.377637 140.000000
10.000000 24.995200 28.739200 29.672320 29.904256 29.961702 29.975870 29.979348 57.471397 140.000000
10.000000 24.999040 28.747648 29.683994 29.917650 29.975870 29.990348 29.993939 57.493067 140.000000
10.000000 24.999808 28.749491 29.686697 29.920869 29.979348 29.993939 29.997576 57.498129 140.000000
10.000000 46.999962 55.149891 56.967318 57.377637 57.471397 57.493067 57.498129 84.999251 140.000000
10.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000 140.000000
Serial - Parallel :
0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 -8.449123 -7.295562 -4.130045 -0.223383 4.700821 11.842837 24.197841 27.989956 0.000000
0.000000 -7.295393 -3.634221 2.625620 9.875105 18.460974 29.734772 46.215059 45.614605 0.000000
0.000000 -4.129553 2.626107 11.768283 21.592233 32.390128 45.252083 61.735559 56.139790 0.000000
0.000000 -0.222550 9.876165 21.592919 33.212395 45.013723 57.874907 72.770581 62.868202 0.000000
0.000000 4.701866 18.462435 32.391369 45.014365 56.813352 68.664552 81.336147 67.717589 0.000000
0.000000 11.843876 29.736295 45.253518 57.875883 68.664974 78.702429 88.756505 71.705275 0.000000
0.000000 24.198650 46.216274 61.736763 72.771486 81.336657 88.756686 95.804935 75.358701 0.000000
0.000000 27.990380 45.615251 56.140444 62.868716 67.717906 71.705418 75.358739 51.429021 0.000000
0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
suppa@HP-Victus-SS:~/Documents/COA project/coa$
```

References :

Jacobi 2D Stencil Algorithm :

- <https://webs.um.es/jmgarcia/miwiki/lib/exe/fetch.php?media=para10.pdf>
- [Vinayaka_MSc_Thesis.pdf](#)

Learning about OpenMP :

- http://www.archer.ac.uk/training/courses/2018/10/openmp_online/index.php
- <https://youtu.be/FjCFT5ojQSk?si=fMcC75YL7opG75Qn>